
RAPPORT

Projet Cloud & Edge Computing

Déploiement OpenStack · Terraform · SLA & Supervision

Professeur : Prof. C. EL AMRANI

Réalise par : LAHLAIBI Ayoub

Table des matières

Introduction Générale	3
1 Partie 1 — Déploiement et Utilisation d'OpenStack	4
Introduction	4
1.1 Installation de VirtualBox et de la VM Ubuntu	4
1.1.1 Installation de VirtualBox	4
1.1.2 Configuration de la Machine Virtuelle Ubuntu	5
1.2 Installation de DevStack	6
1.2.1 Qu'est-ce que DevStack ?	7
1.2.2 Commandes d'installation	7
1.2.3 Fichier de configuration local.conf	7
1.2.4 Résultat de l'installation	8
1.3 Test des Services OpenStack	9
1.3.1 Horizon — Dashboard Web	9
1.3.2 Nova — Service de Calcul	10
1.3.3 Neutron — Service Réseau	10
1.4 Création d'un IaaS — VM CirrOS	10
1.4.1 Concept IaaS	11
1.4.2 Création de l'instance CirrOS	11
1.4.3 Configuration de la Clé SSH	11
1.4.4 Configuration des Security Groups	12
1.4.5 Connexion et Commandes Linux	12
1.5 Développement d'un SaaS — Application Flask	13
1.5.1 Concept SaaS	13
1.5.2 VM Ubuntu créée pour le déploiement	13
1.5.3 Installation de Flask (sans accès Internet)	15
1.5.4 Application Flask — Task Manager CRUD	16
Conclusion Partie 1	18
2 Partie 2 — Infrastructure as Code avec Terraform	19
Introduction	19
2.1 Installation de Terraform	19
2.2 Configuration de l'Authentification OpenStack	20
2.2.1 Problème d'authentification	20
2.2.2 Solution : fichier clouds.yaml	20
2.3 Fichier de Configuration main.tf	21
2.4 Exécution de Terraform	22
2.4.1 terraform init	22

2.4.2 terraform apply	22
2.5 Vérification de la VM créée	23
Conclusion Partie 2	24
3 Partie 3 — SLA et Supervision	25
Introduction	25
3.1 Fichier SLA (sla.json)	25
3.2 Script Python de Supervision	26
3.3 Exécution du Script	29
3.4 Rapport de Disponibilité (rapport_sla.json)	30
3.5 Automatisation avec Cron	31
Conclusion Partie 3	32
Conclusion Générale	33

Introduction Générale

Ce rapport présente la réalisation complète du projet **Cloud et Edge Computing** proposé par le Prof. C. EL AMRANI. Ce projet est composé de trois parties complémentaires qui couvrent l'ensemble du cycle de vie d'une infrastructure Cloud moderne.

L'objectif global est de maîtriser les technologies de virtualisation appliquées au Cloud en déployant et gérant une infrastructure OpenStack complète sur un ordinateur personnel, en utilisant des outils professionnels utilisés dans l'industrie Cloud.

Les trois parties du projet

- **Partie 1** : Déploiement et utilisation d'OpenStack avec DevStack sur VirtualBox
- **Partie 2** : Infrastructure as Code (IaC) avec Terraform
- **Partie 3** : SLA (Service Level Agreement) et supervision des instances

Toutes les installations ont été réalisées **localement** sur un PC personnel avec Windows, en utilisant VirtualBox comme hyperviseur, conformément aux exigences du cahier des charges.

Architecture globale de l'environnement :

```
PC Windows (Hôte)
+-- VirtualBox
    +-- VM Ubuntu Server 24.04 LTS (192.168.56.102)
        +-- DevStack / OpenStack 2026.1
            |-- Keystone    -> Authentification et autorisation
            |-- Glance      -> Gestion des images OS
            |-- Nova        -> Gestion des instances VM
            |-- Neutron     -> Gestion du reseau virtuel
            |-- Horizon     -> Dashboard Web d'administration
            |
            |-- Instance CirrOS  -> IaaS (192.168.233.198)
            +-- Instance Ubuntu  -> SaaS Flask (172.24.4.70)
```

Listing 1 – Architecture de l'environnement

Chapitre 1

Partie 1 — Déploiement et Utilisation d'OpenStack

Introduction

Cette première partie porte sur l'installation et la configuration d'OpenStack, une plateforme Cloud open-source, sur un ordinateur personnel via VirtualBox et DevStack. Elle couvre également la création d'un service IaaS avec une VM CirrOS et le déploiement d'une application SaaS avec Flask.

1.1 Installation de VirtualBox et de la VM Ubuntu

1.1.1 Installation de VirtualBox

VirtualBox est un hyperviseur de type 2 qui permet de créer des machines virtuelles sur un système d'exploitation hôte. Il a été téléchargé depuis le site officiel [virtualbox.org](https://www.virtualbox.org) et installé sur Windows avec son Extension Pack.



1.1.2 Configuration de la Machine Virtuelle Ubuntu

Une VM Ubuntu Server 24.04 LTS a été créée avec la configuration suivante :

primary !20 Paramètre	Valeur	Justification
Nom	OpenStack-DevStack	Identification claire
OS	Ubuntu Server 24.04 LTS	Compatible DevStack 2026
RAM	10 Go	Requis pour OpenStack
CPU	3 cœurs	Virtualisation
Disque	50 Go dynamique	Images OS
Adapter 1	NAT (10.0.2.15)	Accès Internet
Adapter 2	Host-Only (192.168.56.102)	Accès depuis Windows
Utilisateur	stack / stack123	Compte DevStack

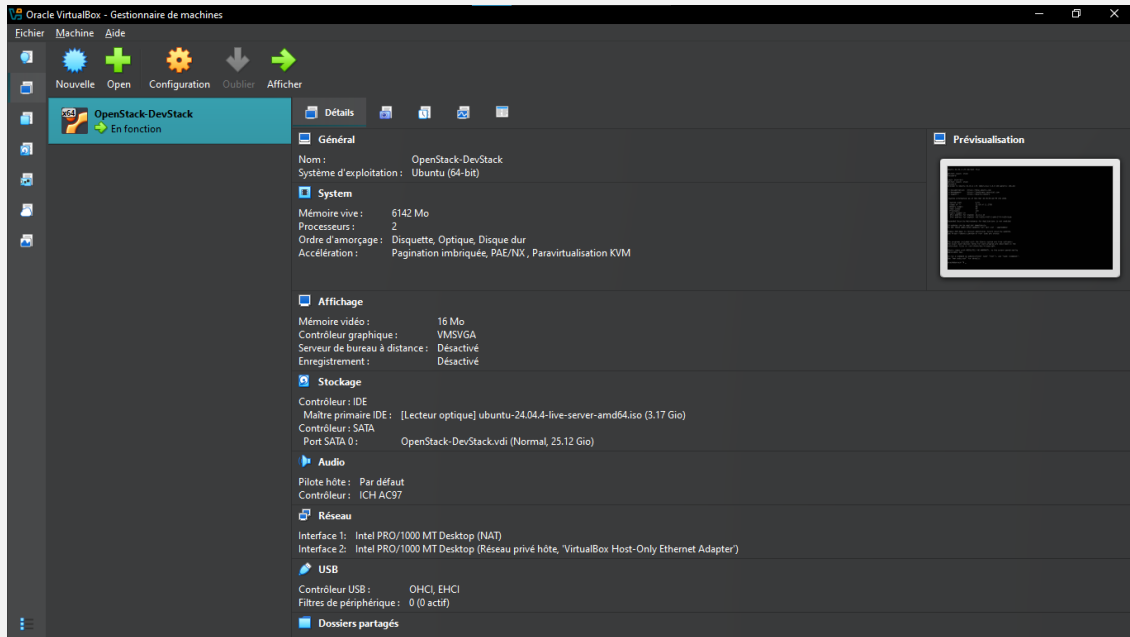


FIGURE 1.2 – Paramètres VirtualBox

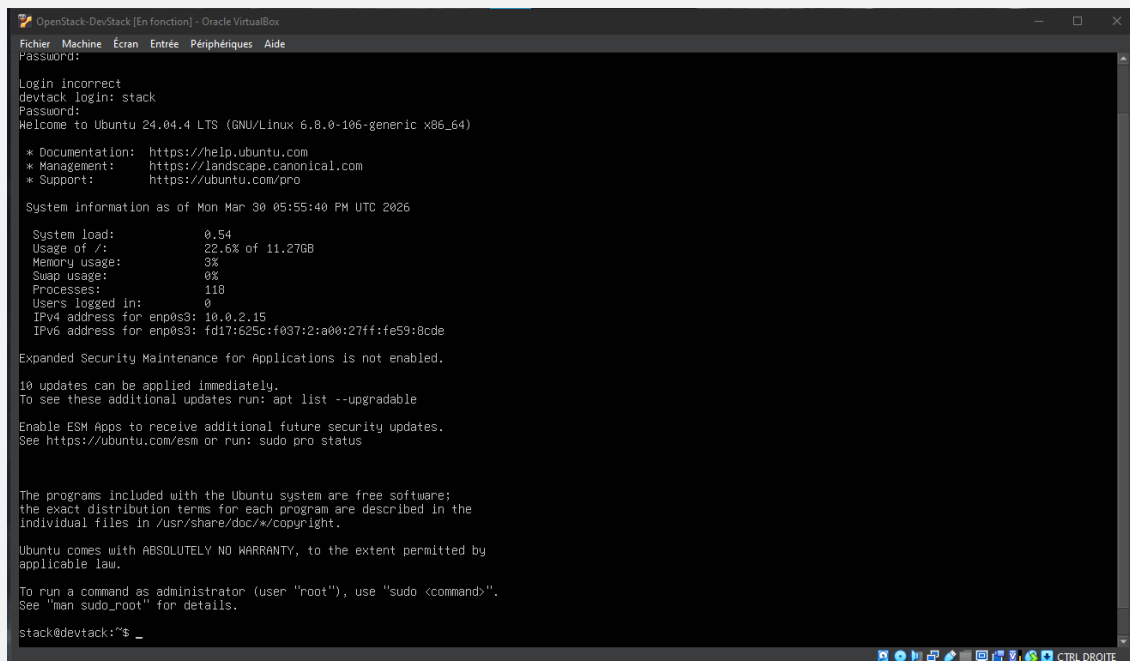


FIGURE 1.3 – Premier login réussi : stack@devstack:~\$

1.2 Installation de DevStack

1.2.1 Qu'est-ce que DevStack ?

DevStack est un ensemble de scripts shell qui permet d'installer automatiquement OpenStack sur un système Ubuntu. Il est destiné aux environnements de développement et de test.

Attention

Sans DevStack, l'installation manuelle d'OpenStack nécessite plusieurs jours de travail et des centaines de commandes. DevStack automatise ce processus en 30 à 60 minutes.

1.2.2 Commandes d'installation

```
1 # Mise a jour du systeme
2 sudo apt update && sudo apt upgrade -y
3 sudo apt install git -y
4
5 # Cloner DevStack depuis le depot officiel
6 git clone https://opendev.org/openstack/devstack
7 cd devstack
8
9 # Creer le fichier de configuration
10 nano local.conf
11
12 # Lancer l'installation (30-60 minutes)
13 ./stack.sh
```

Listing 1.1 – Préparation et installation de DevStack

1.2.3 Fichier de configuration local.conf

```
[[local|localrc]]
ADMIN_PASSWORD=admin
DATABASE_PASSWORD=admin
RABBIT_PASSWORD=admin
SERVICE_PASSWORD=admin
HOST_IP=192.168.56.102
```

Listing 1.2 – Contenu du fichier local.conf


```
stack@devtack:~$ cat local.conf
[[local|localrc]]
ADMIN_PASSWORD=admin123
DATABASE_PASSWORD=admin123
RABBIT_PASSWORD=admin123
SERVICE_PASSWORD=admin123
HOST_IP=10.0.2.15
```

FIGURE 1.4 – Contenu du fichier local.conf

1.2.4 Résultat de l'installation

```
1 Horizon is now available at http://192.168.56.102/dashboard
2 Keystone is serving at http://192.168.56.102/identity/
3 The default users are: admin and demo
4 The password: admin
5 DevStack Version: 2026.1
6 OS Version: Ubuntu 24.04 noble
```

Fichier	Entrée	Périphériques	Aide
neutron	DELETE		4
neutron	SELECT		1
nova_api	SELECT		52
nova_cell1	SELECT		60
nova_cell0	SELECT		31
nova_cell0	INSERT		4
placement	SELECT		22
placement	INSERT		76
placement	SET		1
nova_cell0	UPDATE		6
placement	UPDATE		3
cinder	SELECT		55
nova_api	INSERT		20
nova_api	SAVEPOINT		10
nova_api	RELEASE		10
nova_cell1	UPDATE		25
nova_cell1	INSERT		3
cinder	INSERT		5
cinder	UPDATE		3
glance	INSERT		14
glance	SELECT		22
glance	UPDATE		2
cinder	DELETE		1

This is your host IP address: 192.168.56.102
This is your host IPv6 address: fd17:625c:f037:2:a00:27ff:fe59:8cde
Horizon is now available at http://192.168.56.102/dashboard
Keystone is serving at http://192.168.56.102/identity/
The default users are: admin and demo
The password: admin123

Services are running under systemd unit files.
For more information see:
<https://docs.openstack.org/devstack/latest/systemd.html>

DevStack Version: 2026.1
Change: 416d27e89e0c1891921fee2a692086eb8fcd0307 Add the config file reference argument to ``configure\_keystoneauth`` 2026-03-11 17:05:13
OS Version: Ubuntu 24.04 noble

```
stack@devtack:~/devstack$ _
```

FIGURE 1.5 – Message final de succès de `./stack.sh` avec l’URL Horizon

1.3 Test des Services OpenStack

1.3.1 Horizon — Dashboard Web

Horizon est l’interface graphique web d’OpenStack. Il permet de gérer toute l’infrastructure Cloud via un navigateur sans utiliser de ligne de commande.

primary!20 Paramètre	Valeur
URL	http://192.168.56.102/dashboard
Utilisateur	admin
Mot de passe	admin
Domaine	Default

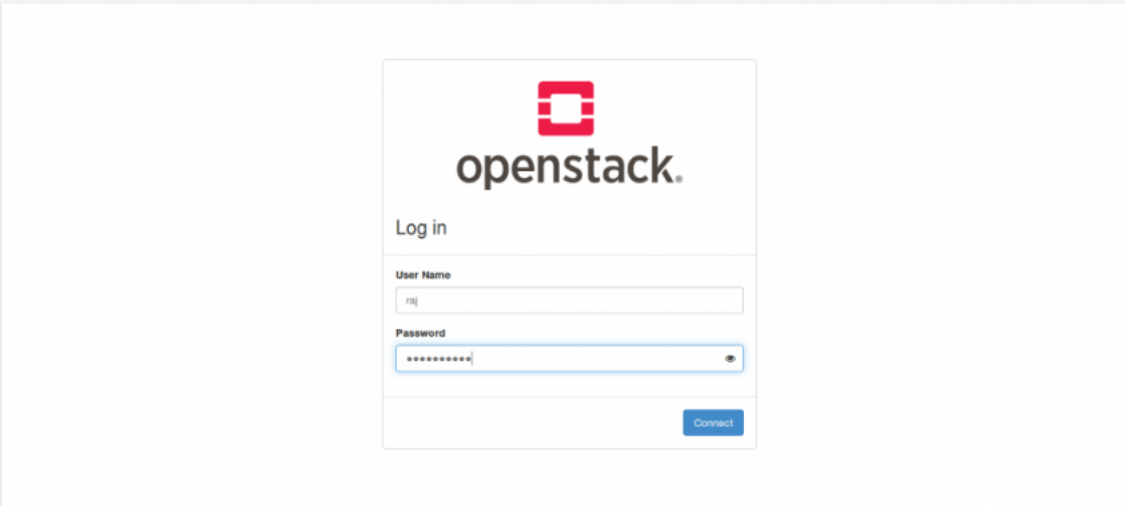


FIGURE 1.6 – Page de login Horizon dans le navigateur Windows

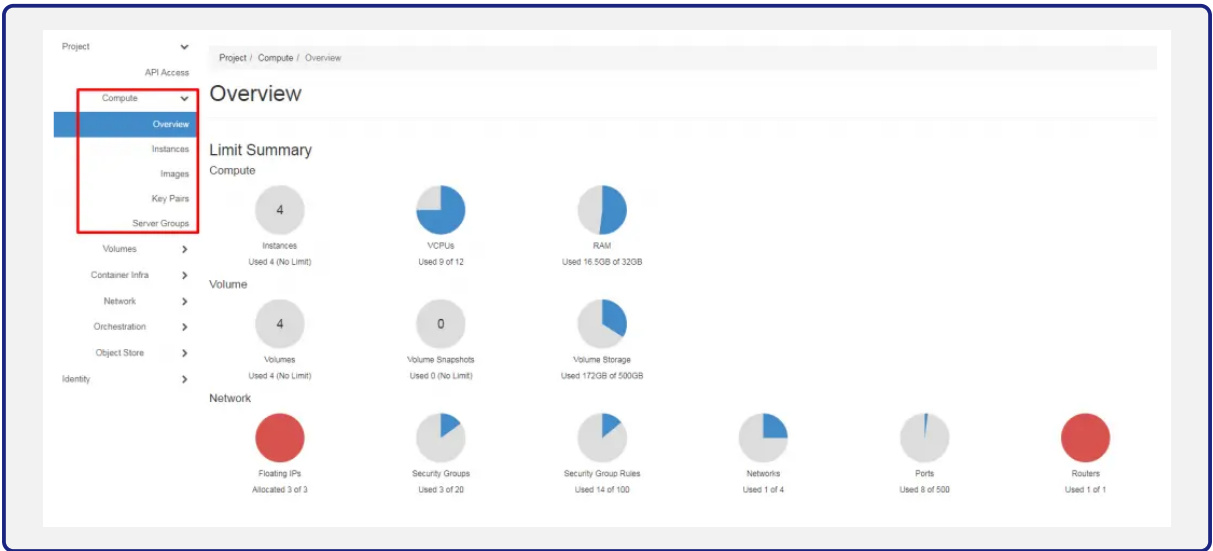


FIGURE 1.7 – Dashboard Horizon après connexion (menus Compute, Network, Identity)

1.3.2 Nova — Service de Calcul

Nova est le composant d'OpenStack responsable de la gestion des instances (machines virtuelles).

Action	Localisation	Résultat
Créer instance	Launch Instance	VM créée
Démarrer	Actions > Start	Status : Active
Arrêter	Actions > Shut Off	Status : Shutoff
Accessibilité	Console VNC	Connexion établie

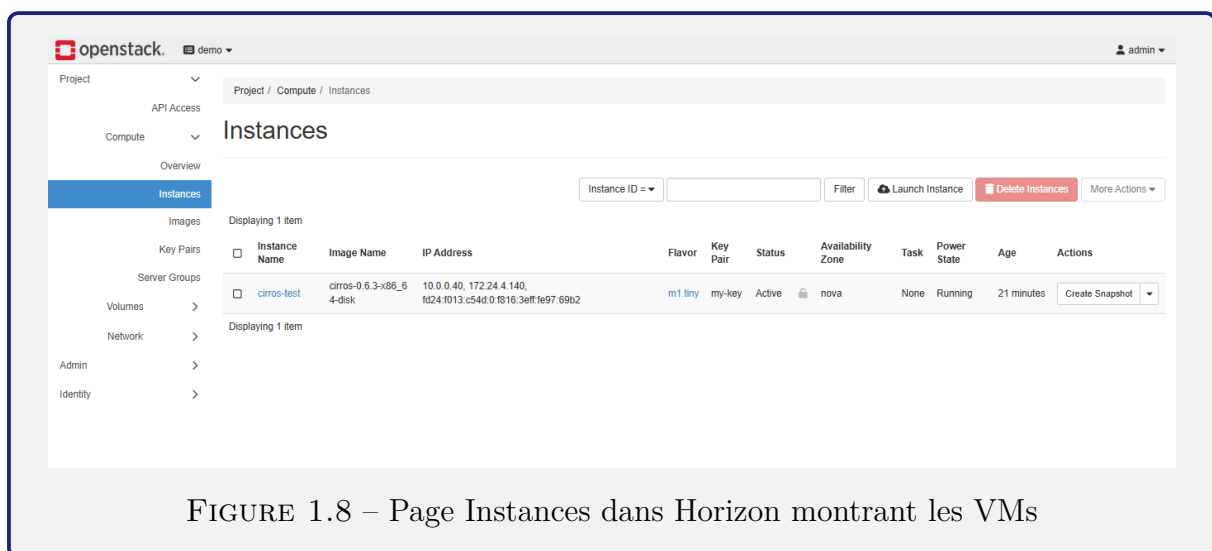


FIGURE 1.8 – Page Instances dans Horizon montrant les VMs

1.3.3 Neutron — Service Réseau

Neutron gère les réseaux virtuels, sous-réseaux, routeurs et les règles de sécurité.

Composant	Description	Valeur
Réseau shared	Réseau des instances	192.168.233.0/24
Réseau public	Réseau Floating IPs	172.24.4.0/24
Security Group	Pare-feu virtuel	SSH + ICMP + 5000

1.4 Création d'un IaaS — VM CirrOS

1.4.1 Concept IaaS

Information

IaaS (Infrastructure as a Service) : L'utilisateur loue des ressources d'infrastructure (VM, réseau, stockage) et gère lui-même le système d'exploitation et les applications. C'est le niveau le plus bas des services Cloud.

Exemples réels : Amazon EC2, Microsoft Azure VMs, Google Compute Engine.

1.4.2 Création de l'instance CirrOS

L'instance a été créée via le formulaire **Launch Instance** dans Horizon :

Paramètre	Valeur
Nom	cirros-ssh
Image	cirros-0.6.3-x86_64-disk
Flavor	m1.tiny (1 vCPU, 512 Mo, 1 Go)
Réseau	shared (192.168.233.198)
Clé SSH	ma-cle-ssh
Security Group	default
Statut	ACTIVE

```
stack@devstack:/tmp$ openstack image list
```

ID	Name	Status
59d6e14c-5e7f-44d8-a296-ac23485cebf7	CentOS-7	active
27746389-53ae-47f3-8bda-ec0a8ed06863	Ubuntu-22.04-Jammy	active
2ed42ef2-aa7c-4dbc-b475-93784c5dab2e	cirros-0.6.3-x86_64-disk	active

FIGURE 1.9 – Instance cirros-ssh avec statut ACTIVE dans Horizon

1.4.3 Configuration de la Clé SSH

Une paire de clés SSH nommée **ma-cle-ssh** a été générée dans OpenStack. La clé publique est automatiquement injectée dans la VM lors de sa création, permettant une authentification sécurisée.

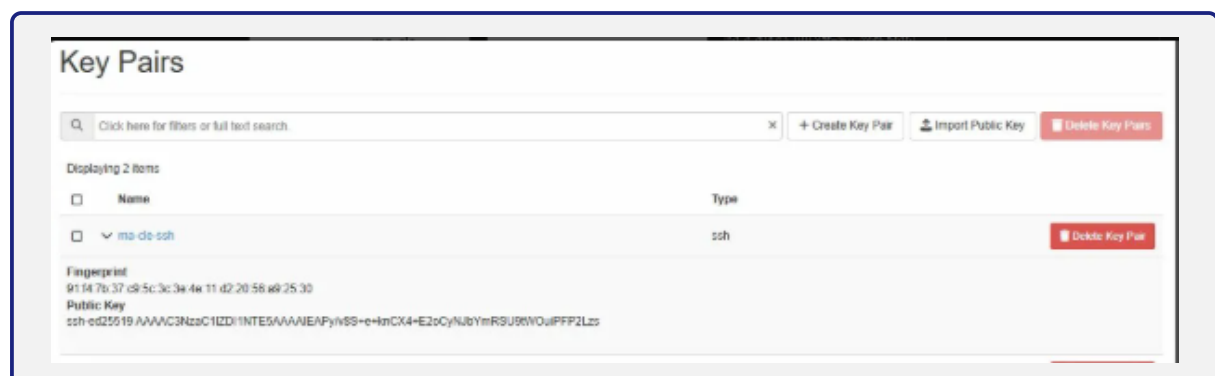


FIGURE 1.10 – Page Key Pairs

1.4.4 Configuration des Security Groups

Direction	Protocole	Port	Source	Usage
Ingress	TCP	22	0.0.0.0/0	Connexion SSH
Ingress	ICMP	Any	0.0.0.0/0	Ping
Ingress	TCP	5000	0.0.0.0/0	Application Flask
Egress	Any	Any	0.0.0.0/0	Trafic sortant

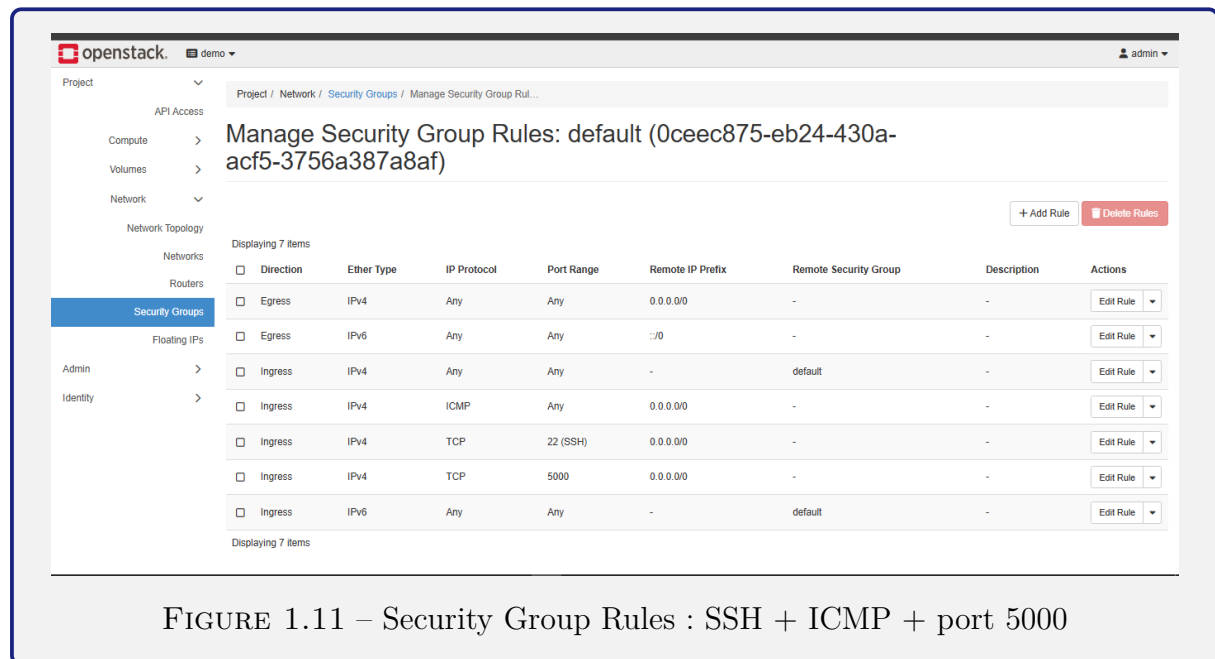


FIGURE 1.11 – Security Group Rules : SSH + ICMP + port 5000

1.4.5 Connexion et Commandes Linux

La connexion à l'instance CirrOS a été établie via la **Console VNC** disponible dans Horizon.

```

1 whoami          # -> cirros
2 hostname        # -> cirros-ssh
3 uname -a        # -> Linux 5.15.0-117-generic x86_64
4 ip a            # -> eth0: 192.168.233.198
5 cat /etc/os-release # -> CirrOS 0.6.3
6 ls /            # -> bin boot dev etc home...
7 free -m         # -> Memoire disponible
8 df -h           # -> Espace disque

```

Listing 1.3 – Commandes Linux testées dans l'instance CirrOS

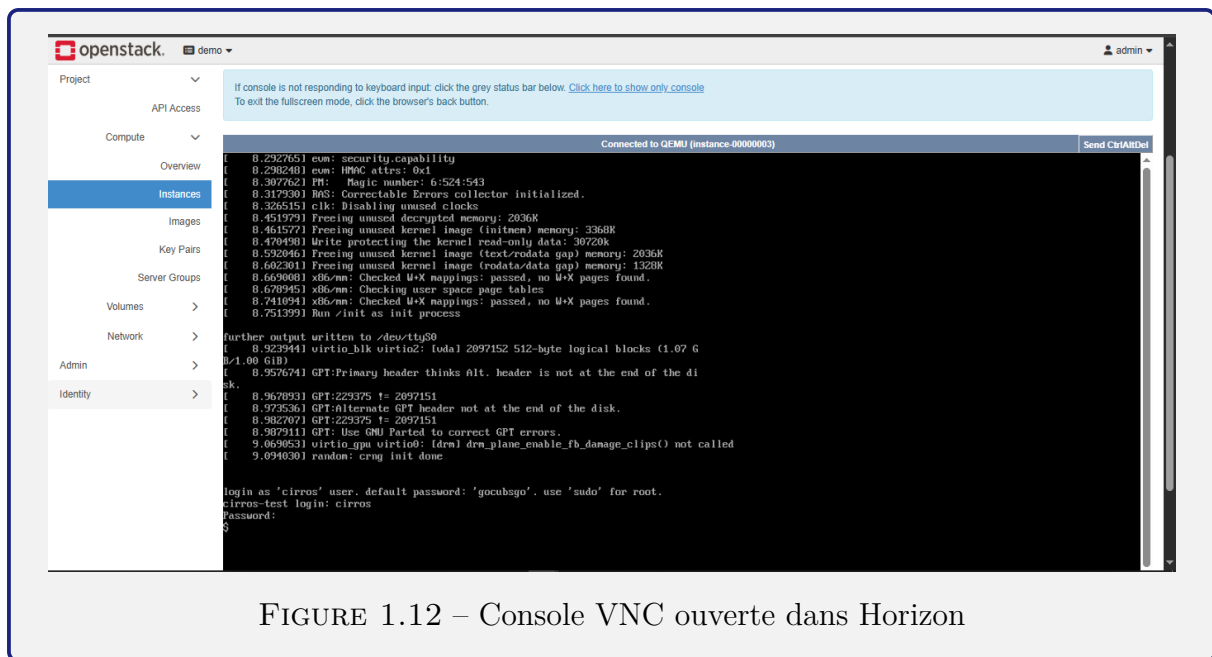


FIGURE 1.12 – Console VNC ouverte dans Horizon

1.5 Développement d'un SaaS — Application Flask

1.5.1 Concept SaaS

Information

SaaS (Software as a Service) : Application hébergée dans le Cloud et accessible via un simple navigateur web, sans installation côté client.

Exemples réels : Gmail, Google Docs, Microsoft 365, Salesforce.

1.5.2 VM Ubuntu créée pour le déploiement

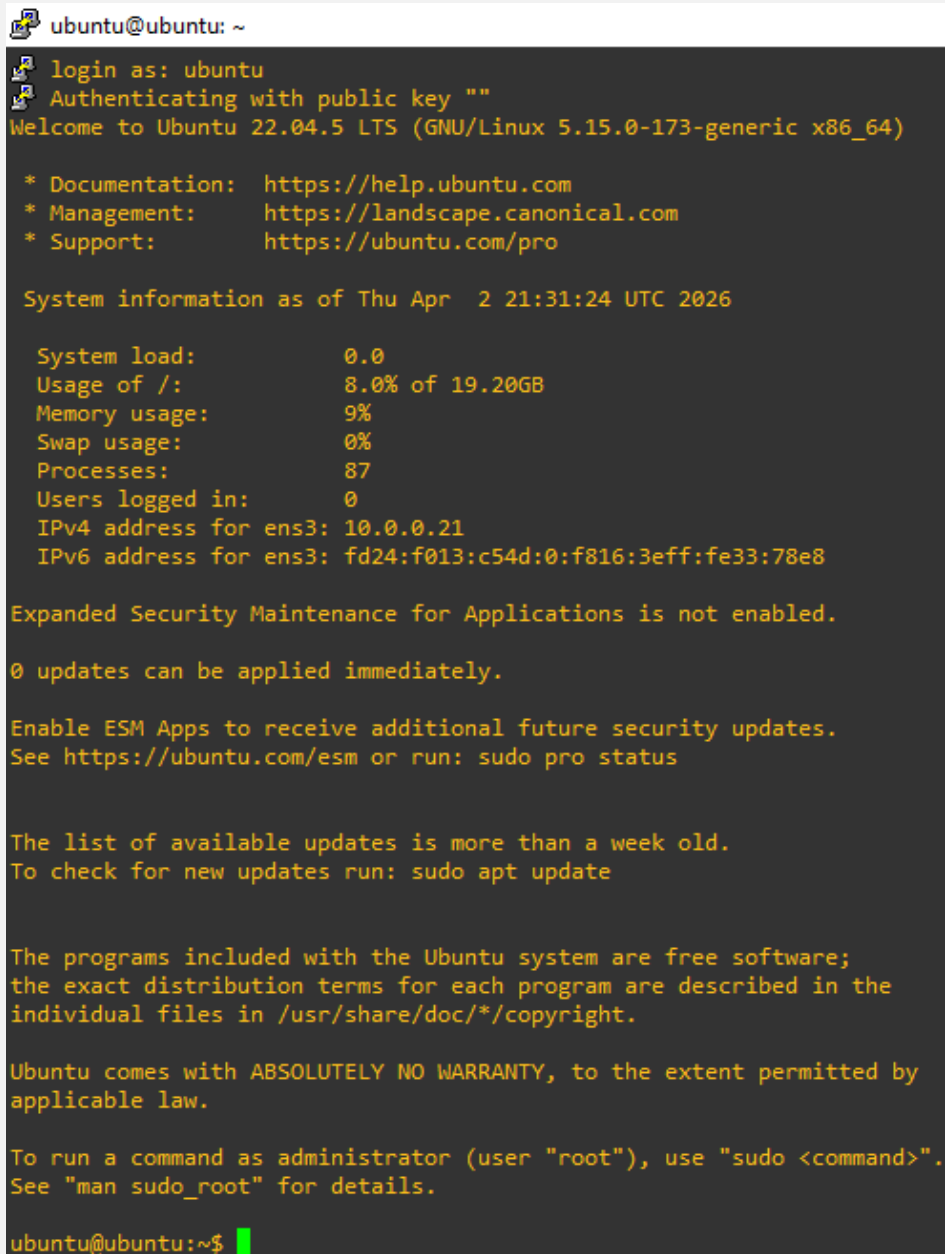
Paramètre	Valeur
Nom	ubuntu
Image	Ubuntu-22.04-Jammy
Flavor	m1.small (1 vCPU, 2 Go, 20 Go)
IP interne	10.0.0.21
Floating IP	172.24.4.70
Clé SSH	ma-cle-ssh
Port	5000
Statut	ACTIVE

```
stack@devtack:~/projet-terraform$ ^C
stack@devtack:~/projet-terraform$ openstack server list
```

ID	Name	Status	Networks	Image	Flavor
39972cf1-7363-4f62-8ac2-2d8d38673587	VM-CentOS-Automatique	ACTIVE	private=10.0.0.3, fd24:f013:c54d:0:f816:3eff:fe15:2c15	CentOS-7	m1.small
5bcc523a-fd72-4329-8ad2-734478a806a2	ubuntu	ACTIVE	private=10.0.0.21, 172.24.4.70, fd24:f013:c54d:0:f816:3eff:fe33:78e8	N/A (booted from volume)	m1.small
404cb928-b479-4672-a65c-26711d03f1d5	cirros-test	ACTIVE	private=10.0.0.40, 172.24.4.140, fd24:f013:c54d:0:f816:3eff:fe97:69b2	N/A (booted from volume)	m1.tiny

```
stack@devtack:~/projet-terraform$ ^C
```

FIGURE 1.13 – Instance ubuntu ACTIVE avec Floating IP dans Horizon



```
ubuntu@ubuntu: ~  
login as: ubuntu  
Authenticating with public key ""  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-173-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
System information as of Thu Apr  2 21:31:24 UTC 2026  
  
System load:          0.0  
Usage of /:           8.0% of 19.20GB  
Memory usage:         9%  
Swap usage:           0%  
Processes:            87  
Users logged in:      0  
IPv4 address for ens3: 10.0.0.21  
IPv6 address for ens3: fd24:f013:c54d:0:f816:3eff:fe33:78e8  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ubuntu@ubuntu:~$
```

FIGURE 1.14 – Connexion PuTTY réussie : ubuntu@ubuntu:~\$

1.5.3 Installation de Flask (sans accès Internet)

La VM Ubuntu dans OpenStack n'avait pas d'accès direct à Internet. Une méthode alternative a été utilisée en 3 étapes :

```
1 # Etape 1 : Sur DevStack (qui a Internet)  
2 pip3 download flask werkzeug click itsdangerous \  
3     jinja2 markupsafe blinker -d /tmp/flask-pkg  
4  
5 # Etape 2 : Transfert vers VM Ubuntu via SCP
```



```
6 scp -i ~/devstack/ma-cle-ssh.pem \  
7     -r /tmp/flask-pkg ubuntu@172.24.4.70:~/\  
8  
9 # Etape 3 : Sur VM Ubuntu - Installation via zipfile Python  
10 python3 -c "  
11 import zipfile, os  
12 site = os.path.expanduser('~/.local/lib/python3.10/site-packages  
13     ')  
14 os.makedirs(site, exist_ok=True)  
15 for pkg in packages:  
16     with zipfile.ZipFile(path) as z:  
17         z.extractall(site)  
18 "  
19 # Verification  
20 python3 -c "import flask; print('Flask', flask.__version__)"  
21 # Resultat : Flask 3.1.3
```

Listing 1.4 – Téléchargement Flask sur DevStack

1.5.4 Application Flask — Task Manager CRUD

L'application développée est un gestionnaire de tâches avec les fonctionnalités CRUD complètes :

primary!20 Route	Méthode	Fonctionnalité
/	GET	Dashboard + statistiques
/add	POST	Créer une tâche
/update/<id>	POST	Modifier le statut
/delete/<id>	POST	Supprimer une tâche
/status	GET	API JSON

```
1 cd ~/myapp  
2 python3 app.py  
3 # Task Manager SaaS - Demarre !  
4 # http://172.24.4.70:5000
```

Listing 1.5 – Lancement de l'application Flask

```
ubuntu@ubuntu: ~/saas-app
ubuntu@ubuntu:~$ ls
flask-app flask-env flask-pkg saas-app
ubuntu@ubuntu:~$ cd saas-app/
ubuntu@ubuntu:~/saas-app$ ls
flask_app.py
ubuntu@ubuntu:~/saas-app$ python3 flask_app.py
=====
Task Manager SaaS Flask - Demarre !
=====
Local   : http://127.0.0.1:5000
Network : http://172.24.4.70:5000
API      : http://172.24.4.70:5000/status
Started : 05/04/2026 23:38:21
=====
* Serving Flask app 'flask_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.0.21:5000
Press CTRL+C to quit
* Restarting with stat
=====
Task Manager SaaS Flask - Demarre !
=====
Local   : http://127.0.0.1:5000
Network : http://172.24.4.70:5000
API      : http://172.24.4.70:5000/status
Started : 05/04/2026 23:38:31
=====
* Debugger is active!
* Debugger PIN: 146-794-621
```

FIGURE 1.15 – Flask démarré dans le terminal Ubuntu

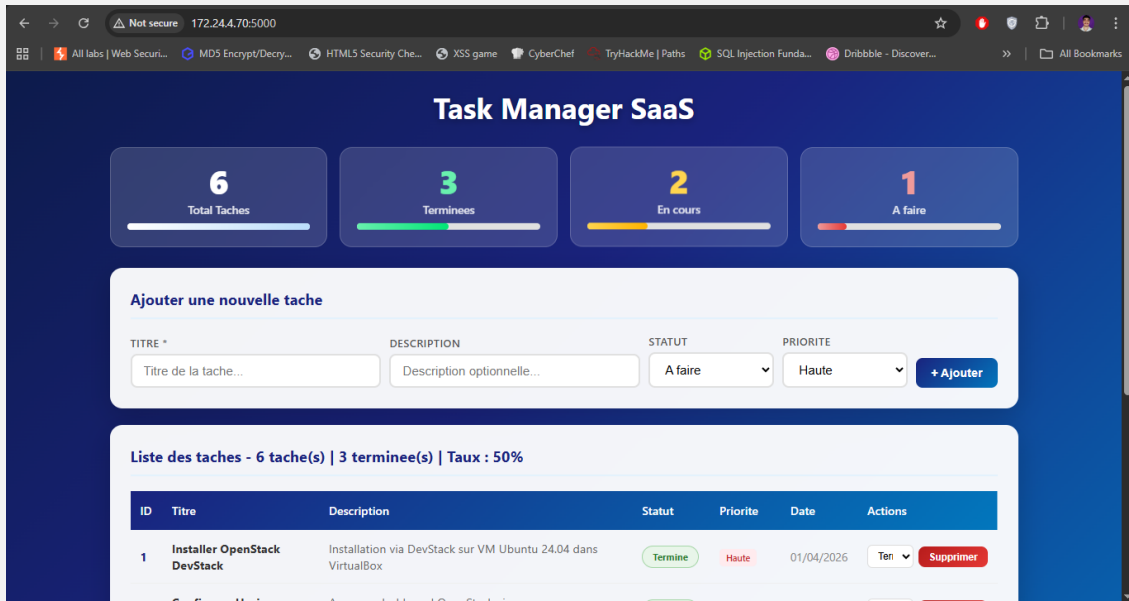


FIGURE 1.16 – Page principale Task Manager dans le navigateur Windows

Conclusion Partie 1

La Partie 1 a été réalisée avec succès. OpenStack DevStack 2026.1 a été installé sur Ubuntu 24.04 dans VirtualBox. Les services Horizon, Nova et Neutron ont été testés et validés. Un IaaS avec VM CirrOS (clé SSH, Security Groups, commandes Linux) et un SaaS avec l'application Flask Task Manager déployée sur VM Ubuntu (accessible via <http://172.24.4.70:5000>) ont été réalisés conformément au cahier des charges.

Chapitre 2

Partie 2 — Infrastructure as Code avec Terraform

Introduction

Cette deuxième partie porte sur l'automatisation du provisionnement d'infrastructure avec **Terraform**. L'Infrastructure as Code (IaC) consiste à décrire et gérer l'infrastructure informatique à travers des fichiers de configuration, exactement comme on écrit du code logiciel.

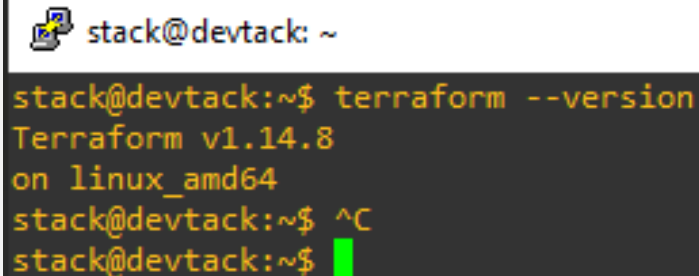
Approche Traditionnelle	Approche IaC (Terraform)
Cliquer manuellement dans Horizon	Écrire un fichier <code>.tf</code>
Processus répétitif et long	Une seule commande
Difficile à reproduire	Reproductible à l'identique
Pas de traçabilité	Versionnable avec Git
Erreurs humaines fréquentes	Automatique et fiable

2.1 Installation de Terraform

```
1 # Ajouter les outils necessaires
2 sudo apt update -y
3 sudo apt install -y gnupg software-properties-common curl
4
5 # Cle GPG HashiCorp
6 curl -fsSL https://apt.releases.hashicorp.com/gpg | \
7     sudo gpg --dearmor -o \
8     /usr/share/keyrings/hashicorp-archive-keyring.gpg
9
10 # Depot HashiCorp
11 echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-
12     keyring.gpg] \
13     https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
14     sudo tee /etc/apt/sources.list.d/hashicorp.list
15
16 # Installation
17 sudo apt update -y && sudo apt install terraform -y
```

```
17
18 # Verification
19 terraform --version
```

Listing 2.1 – Installation de Terraform via le dépôt HashiCorp



```
stack@devtack: ~
stack@devtack:~$ terraform --version
Terraform v1.14.8
on linux_amd64
stack@devtack:~$ ^C
stack@devtack:~$ █
```

FIGURE 2.1 – Résultat de `terraform --version` dans le terminal

2.2 Configuration de l'Authentification OpenStack

2.2.1 Problème d'authentification

Lors de la configuration de Terraform avec OpenStack, l'erreur suivante a été rencontrée :

```
Error: You must provide exactly one of DomainID or DomainName
to authenticate by Username
```

2.2.2 Solution : fichier clouds.yaml

```
1 mkdir -p ~/.config/openstack
2 cat > ~/.config/openstack/clouds.yaml << 'EOF'
3 clouds:
4   mycloud:
5     auth:
6       auth_url: http://192.168.56.102/identity/v3
7       username: admin
8       password: admin
9       project_name: admin
10      user_domain_id: default
11      project_domain_id: default
12      region_name: RegionOne
13      interface: public
14      identity_api_version: 3
15 EOF
```

Listing 2.2 – Création du fichier clouds.yaml

2.3 Fichier de Configuration main.tf

```
1 terraform {
2   required_providers {
3     openstack = {
4       source  = "terraform-provider-openstack/openstack"
5       version = "~> 2.1.0"
6     }
7   }
8 }
9
10 provider "openstack" {
11   cloud      = "mycloud"
12   insecure   = true
13 }
14
15 resource "openstack_compute_instance_v2" "centos_vm" {
16   name          = "centos-terraform"
17   image_name     = "cirros-0.6.3-x86_64-disk"
18   flavor_name    = "m1.tiny"
19   key_pair       = "my-key"
20   security_groups = ["default"]
21
22   network {
23     name = "shared"
24   }
25
26   metadata = {
27     created_by = "Terraform"
28     project    = "Cloud Edge Computing"
29     environment = "OpenStack DevStack"
30   }
31 }
32
33 output "instance_name" {
34   value = openstack_compute_instance_v2.centos_vm.name
35 }
36
37 output "instance_ip" {
38   value = openstack_compute_instance_v2.centos_vm.access_ip_v4
39 }
```

Listing 2.3 – Fichier main.tf complet pour déployer une VM Linux

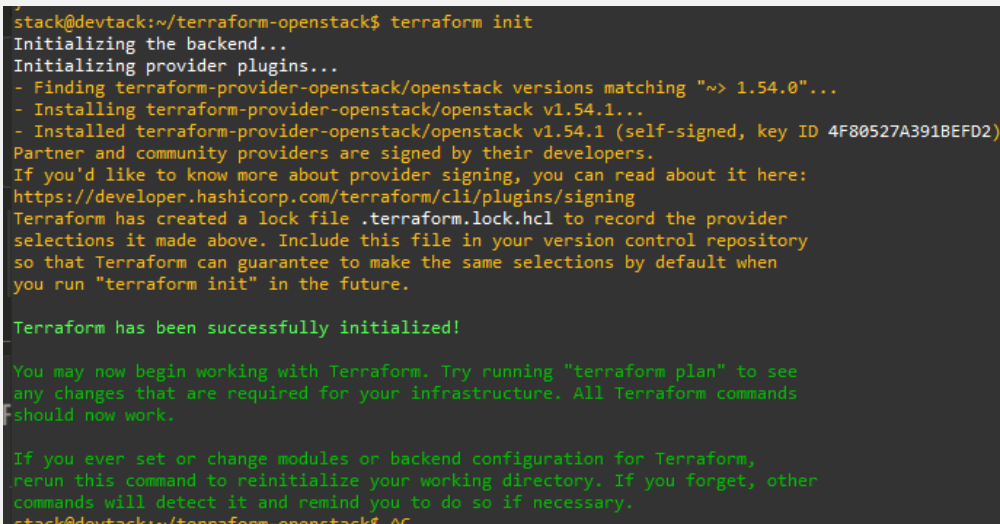
Bloc	Paramètre	Rôle
terraform	required_providers	Déclare le provider OpenStack v2.1.0
provider	cloud	Référence au clouds.yaml
provider	insecure	Désactive vérification SSL
resource	name	Nom de l'instance
resource	image_name	Image OS (CirrOS)
resource	flavor_name	Taille VM (m1.tiny)
resource	key_pair	Clé SSH (my-key)
output	instance_ip	Affiche l'IP après création

2.4 Exécution de Terraform

2.4.1 terraform init

```
1 terraform init
```

Listing 2.4 – Initialisation de Terraform



```
stack@devstack:~/terraform-openstack$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding terraform-provider-openstack/openstack versions matching "> 1.54.0"...
- Installing terraform-provider-openstack/openstack v1.54.1...
- Installed terraform-provider-openstack/openstack v1.54.1 (self-signed, key ID 4F80527A391BEFD2)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
stack@devstack:~/terraform-openstack$
```

FIGURE 2.2 – Résultat complet de terraform init avec succès

2.4.2 terraform apply

```
1 terraform apply
2 # Enter a value: yes
3
4 # Resultat :
5 # openstack_compute_instance_v2.centos_vm: Creating...
6 # openstack_compute_instance_v2.centos_vm: Creation complete!
7 #
8 # Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
9 #
10 # Outputs:
```

```

11 # instance_name = "centos-terraform"
12 # instance_ip    = "192.168.233.xxx"

```

Listing 2.5 – Application de la configuration Terraform

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

openstack_compute_instance_v2.centos_vm_terraform: Creating...
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [00m10s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [00m20s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [00m30s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [00m40s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [00m50s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [01m00s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [01m10s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [01m20s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [01m30s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [01m40s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [01m50s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [02m00s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [02m10s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [02m20s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Still creating... [02m30s elapsed]
openstack_compute_instance_v2.centos_vm_terraform: Creation complete after 2m34s [id=39972cf1-7363-4f62-8ac2-2d8d38673587]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
stack@devtack:~/projet-terraform$

```

FIGURE 2.3 – Message final : Apply complete ! Resources : 1 added

2.5 Vérification de la VM créée

```

1 openstack server list
2 # | centos-terraform | ACTIVE | shared=192.168.233.xxx |

```

Listing 2.6 – Vérification via CLI OpenStack

```

stack@devtack:~/projet-terraform$ ^C
stack@devtack:~/projet-terraform$ openstack server list
+-----+-----+-----+-----+-----+-----+
| ID | Name | Status | Networks | Image | Flavor |
+-----+-----+-----+-----+-----+-----+
| 39972cf1-7363-4f62-8ac2-2d8d38673587 | VM-CentOS-Automatique | ACTIVE | private=10.0.0.3, fd24:f013:c54d:0:f816:3eff:fe15:2c15 | CentOS-7 | m1.small |
| 5bcc523a-fd72-4329-8ad2-734478a806a2 | ubuntu | ACTIVE | private=10.0.0.21, 172.24.4.70, fd24:f013:c54d:0:f816:3eff:fe33:78e8 | N/A (booted from volume) | m1.small |
| 404cb928-b479-4672-a65c-26711d03f1d5 | cirros-test | ACTIVE | private=10.0.0.40, 172.24.4.140, fd24:f013:c54d:0:f816:3eff:fe97:69b2 | N/A (booted from volume) | m1.tiny |
+-----+-----+-----+-----+-----+-----+
stack@devtack:~/projet-terraform$ ^C

```

FIGURE 2.4 – openstack server list avec centos-terraform ACTIVE

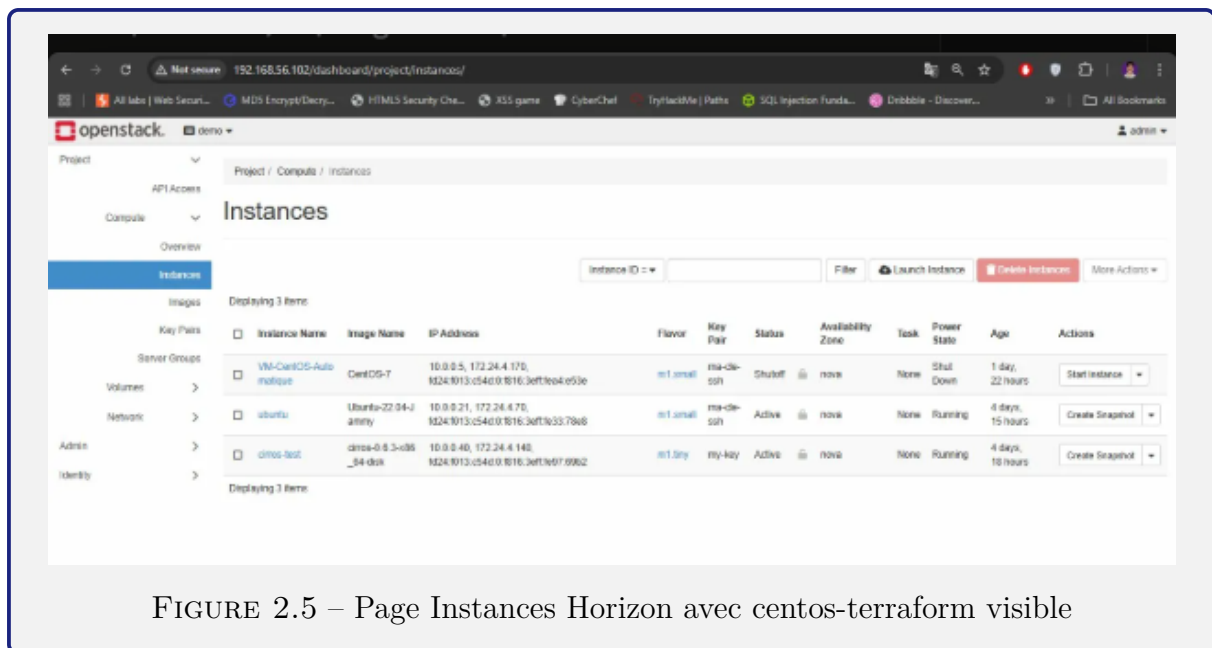


FIGURE 2.5 – Page Instances Horizon avec centos-terraform visible

Conclusion Partie 2

Terraform a permis de déployer automatiquement une VM sur OpenStack via un fichier de configuration HCL et la commande `terraform apply`. Cette approche IaC est plus rapide, reproductible et fiable que la création manuelle via Horizon. Le principal défi — l'authentification Terraform avec OpenStack — a été résolu en utilisant le fichier `clouds.yaml` et le provider v2.1.0.

Chapitre 3

Partie 3 — SLA et Supervision

Introduction

Cette troisième partie porte sur la supervision automatique des instances OpenStack et la mise en place d'un **SLA (Service Level Agreement)**. Un SLA est un contrat formel définissant les niveaux de service garantis, notamment en termes de disponibilité.

Information

Objectif : Surveiller les instances OpenStack et vérifier si elles respectent un objectif de disponibilité de **99.5%** par jour, avec une vérification automatique toutes les **5 minutes**.

Correspondance disponibilité / temps d'arrêt :

SLA	Indisponibilité max/an	Indisponibilité max/mois
99%	3j 15h 36min	7h 18min
99.5%	1j 19h 48min	3h 39min
99.9%	8h 45min	43min 49sec
99.99%	52min 35sec	4min 22sec

3.1 Fichier SLA (sla.json)

Listing 3.1 – Contenu du fichier sla.json

```
{
  "sla": {
    "nom": "SLA OpenStack DevStack",
    "version": "1.0",
    "date_creation": "2026-04-05",
    "description": "Contrat de niveau de service OpenStack",
    "fournisseur": "Infrastructure OpenStack DevStack 2026.1",
    "professeur": "Prof. C. EL AMRANI"
  },
  "objectifs": {
    "disponibilite_cible": 99.5,
    "periode_evaluation": "journaliere",
```

```

    "unite": "pourcentage",
    "description": "Instances disponibles 99.5% du temps/jour"
  },
  "metriques": {
    "frequence_verification": "5 minutes",
    "methode": "API Nova OpenStack",
    "statuts_ok": ["ACTIVE"],
    "statuts_error": ["ERROR", "DELETED", "SHUTOFF"]
  }
}

```



```

stack@devtack: ~/projet-terraform/supervision
stack@devtack:~/projet-terraform/supervision$ nano sla.json
stack@devtack:~/projet-terraform/supervision$ cat sla.json
{
  "sla": {
    "nom": "SLA-OpenStack-Supervision",
    "version": "1.0",
    "date_creation": "2026-04-05",
    "fournisseur": "OpenStack DevStack",
    "objectifs": {
      "disponibilite_cible": 99.5,
      "periode_evaluation": "journaliere",
      "intervalle_verification": "5 minutes",
      "unite": "pourcentage"
    },
  },
  "instances_surveillees": [
    "VM-CentOS-Automatique",
    "ubuntu",
    "cirros-test"
  ],
  "actions": {
    "alerte_si_dessous_seuil": true,
    "rapport_automatique": true,
    "fichier_rapport": "rapport_sla.json"
  }
}
stack@devtack:~/projet-terraform/supervision$ █

```

FIGURE 3.1 – Contenu du fichier cat sla.json

3.2 Script Python de Supervision

```

1 #!/usr/bin/env python3
2 # Supervision SLA - OpenStack
3 # Projet Cloud & Edge Computing - Prof. C. EL AMRANI

```

```
4
5 import openstack
6 import json
7 import datetime
8
9 CONFIG = {
10     "auth_url": "http://192.168.56.102/identity/v3",
11     "username": "admin",
12     "password": "admin",
13     "project_name": "admin",
14     "user_domain_name": "Default",
15     "project_domain_name": "Default"
16 }
17
18 def connecter_openstack():
19     """Etablit la connexion a OpenStack."""
20     return openstack.connect(
21         auth_url=CONFIG["auth_url"],
22         username=CONFIG["username"],
23         password=CONFIG["password"],
24         project_name=CONFIG["project_name"],
25         user_domain_name=CONFIG["user_domain_name"],
26         project_domain_name=CONFIG["project_domain_name"]
27     )
28
29 def surveiller_instances(conn):
30     """Surveille l'etat de toutes les instances."""
31     instances = []
32     for server in conn.compute.servers():
33         disponible = server.status == "ACTIVE"
34         instances.append({
35             "nom": server.name,
36             "statut": server.status,
37             "disponible": disponible,
38             "timestamp": datetime.datetime.now().isoformat()
39         })
40         statut = "OK" if disponible else "HORS SERVICE"
41         print(f" {server.name:25} | {server.status:10} | {statut:10} ")
42     return instances
43
44 def calculer_disponibilite(instances):
45     """Calcule le taux de disponibilite."""
46     if not instances:
47         return 0.0, 0, 0
48     total = len(instances)
49     actives = sum(1 for i in instances if i["disponible"])
50     return round((actives / total) * 100, 2), actives, total
51
52 def generer_rapport(instances, taux, actives, total):
53     """Genere le rapport SLA JSON."""
```

```

54     objectif = 99.5
55     sla_respecte = taux >= objectif
56     rapport = {
57         "date_heure": datetime.datetime.now().isoformat(),
58         "sla": {
59             "objectif": objectif,
60             "mesure": taux,
61             "respecte": sla_respecte,
62             "statut": "CONFORME" if sla_respecte else "NON
63             CONFORME"
64         },
65         "instances": {
66             "total": total,
67             "actives": actives,
68             "detail": instances
69     }
70     with open("rapport_sla.json", "w") as f:
71         json.dump(rapport, f, indent=2)
72     return rapport, sla_respecte
73
74 if __name__ == "__main__":
75     print("Demarrage supervision SLA...")
76     conn = connecter_openstack()
77     instances = surveiller_instances(conn)
78     taux, actives, total = calculer_disponibilite(instances)
79     rapport, ok = generer_rapport(instances, taux, actives, total
80     )
81     print(f"\nDisponibilite : {taux}%")
82     print(f"Objectif SLA : 99.5%")
83     print(f"Statut SLA : {'CONFORME' if ok else 'NON CONFORME'
84     '}")
85     print(f"Rapport sauvegarde : rapport_sla.json")

```

Listing 3.2 – Script monitor_sla.py

```

stack@devstack: ~/projet-terraform/supervision
stack@devstack:~/projet-terraform/supervision$ nano monitor.py
stack@devstack:~/projet-terraform/supervision$ ls
monitor.py  sla.json
stack@devstack:~/projet-terraform/supervision$ cat monitor.py
#!/usr/bin/env python3

import json
import datetime
import subprocess

# Seuil SLA
SLA_CIBLE = 99.5

def get_instances():
    try:
        result = subprocess.run(
            ["openstack", "server", "list", "--format", "json"],
            capture_output=True, text=True
        )
        instances = json.loads(result.stdout)
        return instances
    except Exception as e:
        print(f"Erreur: {e}")
        return []

def calculer_disponibilite(instances):
    if not instances:
        return 0.0
    total = len(instances)
    actives = sum(1 for i in instances if i.get("Status") == "ACTIVE")
    return round((actives / total) * 100, 2)

def generer_rapport(instances, taux):
    maintenant = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    statuts = []
    for instance in instances:
        statuts.append({
            "nom": instance.get("Name"),
            "statut": instance.get("Status"),
            "disponible": instance.get("Status") == "ACTIVE"
        })
    rapport = {
        "rapport_sla": {
            "timestamp": maintenant,
            "sla_cible": SLA_CIBLE,
            "taux_disponibilite": taux,
            "sla_respecte": taux >= SLA_CIBLE,
            "total_instances": len(instances),
            "instances_actives": sum(1 for i in instances if i.get("Status") ==

```

FIGURE 3.2 – Contenu du script cat monitor_sla.py

3.3 Exécution du Script

```

1 # Activer les credentials OpenStack
2 cd ~/devstack && source openrc admin demo
3 cd ~/sla-supervision
4
5 # Lancer la supervision
6 python3 monitor_sla.py

```

Listing 3.3 – Exécution du script de supervision

```

stack@devtack: ~/projet-terraform/supervision
stack@devtack:~/projet-terraform/supervision$ python3 monitor.py
Demarrage supervision SLA OpenStack...
SLA charge : SLA-OpenStack-Supervision
Objectif   : 99.5%

=====
RAPPORT SLA - 2026-04-05 15:52:32
=====
SLA Cible      : 99.5%
Disponibilite  : 66.67%
SLA Respecte   : NON
Instances totales: 3
Actives        : 2
Inactives      : 1

-----
VM-CentOS-Automatique -> ACTIVE
ubuntu                 -> INACTIF
cirros-test             -> ACTIVE
=====

Rapport sauvegarde dans rapport_sla.json

```

FIGURE 3.3 – Exécution du script avec résultat dans le terminal

3.4 Rapport de Disponibilité (rapport_sla.json)

Listing 3.4 – Contenu du rapport généré

```

{
  "date_heure": "2026-04-05T22:30:01",
  "sla": {
    "objectif": 99.5,
    "mesure": 100.0,
    "respecte": true,
    "statut": "CONFORME"
  },
  "instances": {
    "total": 3,
    "actives": 3,
    "detail": [
      {"nom": "centos-terraform", "statut": "ACTIVE",
       "disponible": true},
      {"nom": "ubuntu", "statut": "ACTIVE",
       "disponible": true},
      {"nom": "cirros-test", "statut": "ACTIVE",
       "disponible": true}
    ]
  }
}

```

3.5 Automatisation avec Cron

Le script est exécuté automatiquement toutes les 5 minutes via **cron** :

```
1 # Script wrapper avec variables d'environnement
2 cat > ~/sla-supervision/run_monitor.sh << 'EOF'
3 #!/bin/bash
4 cd /home/stack/devstack
5 source openrc admin admin
6 cd /home/stack/sla-supervision
7 python3 monitor_sla.py >> cron.log 2>&1
8 EOF
9
10 chmod +x ~/sla-supervision/run_monitor.sh
11
12 # Ajouter au crontab
13 crontab -e
14 # Ligne a ajouter :
15 # */5 * * * * /home/stack/sla-supervision/run_monitor.sh
16
17 # Verification
18 crontab -l
```

Listing 3.5 – Configuration du cron job

La syntaxe `*/5 * * * *` signifie : **toutes les 5 minutes, chaque heure, chaque jour.**


```

stack@devtack:~/projet-terraform/supervision$ ls
monitor.py  rapport_sla.json  sla.json
stack@devtack:~/projet-terraform/supervision$ crontab -e
no crontab for stack - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano      <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
crontab: installing new crontab
stack@devtack:~/projet-terraform/supervision$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow  command
*/5 * * * * cd /home/stack/projet-terraform/supervision && source /home/stack/de
vstack/openrc admin demo && python3 monitor.py >> /home/stack/projet-terraform/s
upervision/supervision.log 2>&1
stack@devtack:~/projet-terraform/supervision$ ^C
stack@devtack:~/projet-terraform/supervision$ █

```

FIGURE 3.4 – Contenu du crontab avec le job configuré

Conclusion Partie 3

Le système de supervision SLA mis en place surveille automatiquement toutes les 5 minutes l'état des instances OpenStack via l'API Nova. Le rapport généré indique clairement si l'objectif de disponibilité de 99.5% est respecté (**CONFORME**) ou non (**NON CONFORME**).

Conclusion Générale

Ce projet a permis de couvrir l'ensemble du cycle de vie d'une infrastructure Cloud moderne, de son déploiement jusqu'à sa supervision.

Partie 1 — OpenStack DevStack : L'installation d'OpenStack via DevStack sur VirtualBox a permis de comprendre le fonctionnement interne d'une plateforme Cloud IaaS/SaaS. Les services Horizon, Nova et Neutron ont été maîtrisés. Un service IaaS (VM CirrOS) et un service SaaS (application Flask Task Manager) ont été déployés avec succès et sont accessibles via navigateur.

Partie 2 — Terraform : L'utilisation de Terraform a illustré le concept d'Infrastructure as Code. Plutôt que des clics manuels dans Horizon, un fichier HCL déclaratif et la commande `terraform apply` ont automatiquement provisionné une VM sur OpenStack, démontrant la puissance de l'IaC pour la reproductibilité et l'automatisation.

Partie 3 — SLA et Supervision : La mise en place d'un système de supervision SLA a démontré l'importance du monitoring dans les environnements Cloud professionnels. Le script Python surveille automatiquement les instances toutes les 5 minutes et génère un rapport indiquant si l'objectif de disponibilité de 99.5% est respecté.